

Java Synchronization Methods: Detailed Explanation

Synchronization in Java is a mechanism that ensures that only one thread can access a shared resource at a time. This is crucial in multi-threaded applications to prevent data inconsistency and corruption. Below is a comprehensive overview of Java synchronization methods, including key concepts, types of synchronization, and examples.

1. Why Use Synchronization?

- **Prevent Thread Interference:** When multiple threads access shared resources, synchronization helps prevent conflicts and ensures that the data remains consistent.
 - **Avoid Concurrency Problems:** Issues like race conditions, where the outcome depends on the sequence or timing of uncontrollable events, can be mitigated through synchronization.
-

2. Types of Synchronization

Java provides two primary types of synchronization:

1. **Method Synchronization**
2. **Block Synchronization**

2.1 Method Synchronization

- **Synchronized Methods:** You can declare a method as synchronized by adding the `synchronized` keyword to its declaration. This ensures that only one thread can execute this method on the same object at a time.

Example:

```
java
VerifyOpen In EditorRunCopy code
1class SynchronizedCounter {
2    private int count = 0;
3
4    public synchronized void increment() {
5        count++;
6    }
7
8    public synchronized void decrement() {
9        count--;
10    }
11
12    public synchronized int value() {
13        return count;
14    }
15}
16
17public class TestSynchronization {
18    public static void main(String[] args) {
19        SynchronizedCounter counter = new SynchronizedCounter();
20        // Create threads that increment and decrement the counter
21    }
22}
```

- **Key Points:**
 - When a thread is executing a synchronized method, all other threads that invoke synchronized methods on the same object will block until the first thread completes.
 - This establishes a happens-before relationship, ensuring visibility of changes made by one thread to others.

2.2 Block Synchronization

- **Synchronized Blocks:** Instead of synchronizing the entire method, you can synchronize a specific block of code within a method. This is more efficient as it allows finer control over the synchronization.

Example:

```
java
VerifyOpen In EditorRunCopy code
1class Counter {
2    private int count = 0;
3
4    public void increment() {
5        synchronized (this) { // Synchronizing a block
6            count++;
7        }
8    }
9
10   public int value() {
11       return count;
12   }
13}
```

- **Key Points:**
 - You can synchronize on any object, not just `this`. For example, you can synchronize on a class-level lock or another object.
 - This allows for more granular control over which parts of your code are synchronized.
-

3. Static Synchronization

- **Static Synchronized Methods:** When you declare a static method as synchronized, the lock is associated with the class itself rather than an instance of the class. This means that only one thread can execute any static synchronized method of that class at a time.

Example:

```
java
VerifyOpen In EditorRunCopy code
1class StaticCounter {
2    private static int count = 0;
3
4    public static synchronized void increment() {
5        count++;
6    }
7
8    public static int value() {
9        return count;
10   }
```

11}

- **Key Points:**
 - Static synchronization is useful when you want to control access to class-level data.
 - It prevents multiple threads from executing any static synchronized method simultaneously.
-

4. Inter-Thread Communication

Java also provides methods for inter-thread communication, which allows threads to communicate with each other. The key methods are:

- **wait():** Causes the current thread to wait until another thread invokes the `notify()` or `notifyAll()` method for the same object.
- **notify():** Wakes up a single thread that is waiting on the object's monitor.
- **notifyAll():** Wakes up all threads that are waiting on the object's monitor.

Example:

```
java
VerifyOpen In EditorRunCopy code
1class SharedResource {
2    private int data;
3
4    public synchronized void produce(int value) {
5        data = value;
6        notify(); // Notify waiting threads
7    }
8
9    public synchronized int consume() throws InterruptedException {
10        while (data == 0) {
11            wait(); // Wait until data is produced
12        }
13        return data;
14    }
15}
```

- **Key Points:**
 - `wait()`, `notify()`, and `notifyAll()` must be called from within a synchronized context.
 - These methods help in reducing CPU usage by avoiding busy-waiting.
-

5. Drawbacks of Synchronization

- **Performance Overhead:** Synchronization can lead to performance bottlenecks, especially if many threads are contending for the same lock.
 - **Deadlocks:** Improper use of synchronization can lead to deadlocks, where two or more threads are waiting indefinitely for each other to release locks.
-

Conclusion

Java synchronization is a powerful tool for managing access to shared resources in multi-threaded applications. By understanding the different synchronization methods and their appropriate use cases, you can write safer and more efficient concurrent programs. If you have any further questions or need clarification on specific topics, feel free to ask!

Java `wait()`, `notify()`, and `notifyAll()` Methods

In Java, the `wait()`, `notify()`, and `notifyAll()` methods are essential for inter-thread communication. They are defined in the `Object` class and are used to manage the execution of threads that need to wait for specific conditions or events. Below is a detailed explanation of each method, along with examples to illustrate their usage.

1. Overview of `wait()`, `notify()`, and `notifyAll()`

- **`wait()`**: Causes the current thread to wait until another thread invokes `notify()` or `notifyAll()` on the same object. The thread releases the lock on the object and enters the waiting state.
- **`notify()`**: Wakes up a single thread that is waiting on the object's monitor. If multiple threads are waiting, one of them is chosen at the discretion of the thread scheduler.
- **`notifyAll()`**: Wakes up all threads that are waiting on the object's monitor. All waiting threads will compete for the lock once it is released.

2. Key Points

- **Synchronized Context**: These methods must be called from a synchronized context (i.e., within a synchronized block or method) to ensure that the calling thread holds the object's monitor.
- **`IllegalMonitorStateException`**: If a thread calls `wait()`, `notify()`, or `notifyAll()` without holding the object's monitor, it throws this exception.
- **Spurious Wakeups**: Threads may wake up without being notified. Therefore, it is common to call `wait()` in a loop that checks the condition.

3. Example Implementation

3.1 Message Class

This class will be used to share messages between threads.

```
java
VerifyOpen In EditorRunCopy code
1public class Message {
2    private String msg;
3
4    public Message(String str) {
5        this.msg = str;
6    }
7}
```

```

8    public String getMsg() {
9        return msg;
10   }
11
12   public void setMsg(String str) {
13       this.msg = str;
14   }
15}

```

3.2 Waiter Class

This class represents a thread that waits for a message to be set.

```

java
VerifyOpen In EditorRunCopy code
1public class Waiter implements Runnable {
2    private Message msg;
3
4    public Waiter(Message m) {
5        this.msg = m;
6    }
7
8    @Override
9    public void run() {
10        String name = Thread.currentThread().getName();
11        synchronized (msg) {
12            try {
13                System.out.println(name + " waiting to get notified at time: "
+ System.currentTimeMillis());
14                msg.wait(); // Wait for notification
15            } catch (InterruptedException e) {
16                e.printStackTrace();
17            }
18            System.out.println(name + " waiter thread got notified at time: "
+ System.currentTimeMillis());
19            // Process the message now
20            System.out.println(name + " processed: " + msg.getMsg());
21        }
22    }
23}

```

3.3 Notifier Class

This class represents a thread that sets a message and notifies waiting threads.

```

java
VerifyOpen In EditorRunCopy code
1public class Notifier implements Runnable {
2    private Message msg;
3
4    public Notifier(Message msg) {
5        this.msg = msg;
6    }
7
8    @Override
9    public void run() {
10        String name = Thread.currentThread().getName();
11        System.out.println(name + " started");
12        try {
13            Thread.sleep(1000); // Simulate some work
14            synchronized (msg) {
15                msg.setMsg(name + " Notifier work done");
16                msg.notify(); // Notify one waiting thread

```

```

17         // msg.notifyAll(); // Uncomment to notify all waiting threads
18     }
19 } catch (InterruptedException e) {
20     e.printStackTrace();
21 }
22 }
23}

```

3.4 Test Class

This class creates and starts the threads.

```

java
VerifyOpen In EditorRunCopy code
1public class WaitNotifyTest {
2    public static void main(String[] args) {
3        Message msg = new Message("process it");
4        Waiter waiter1 = new Waiter(msg);
5        new Thread(waiter1, "Waiter1").start();
6
7        Waiter waiter2 = new Waiter(msg);
8        new Thread(waiter2, "Waiter2").start();
9
10       Notifier notifier = new Notifier(msg);
11       new Thread(notifier, "Notifier").start();
12
13       System.out.println("All the threads are started");
14   }
15}

```

4. Output Explanation

When you run the `WaitNotifyTest` class, you will see output similar to the following:

```

VerifyOpen In EditorRunCopy code
1All the threads are started
2Waiter1 waiting to get notified at time: 1234567890123
3Waiter2 waiting to get notified at time: 1234567890124
4Notifier started
5Waiter1 waiter thread got notified at time: 1234567891123
6Waiter1 processed: Notifier work done

```

- **Waiter Threads:** Both `Waiter1` and `Waiter2` are waiting for a notification.
- **Notifier Thread:** After a delay, the `Notifier` sets the message and calls `notify()`, waking up one of the waiting threads (in this case, `Waiter1`).

5. Differences Between `notify()` and `notifyAll()`

- **`notify()`:** Wakes up a single waiting thread. If multiple threads are waiting, it is non-deterministic which thread will be awakened.
- **`notifyAll()`:** Wakes up all waiting threads. Each thread will then compete for the lock on the object.

6. Conclusion

The `wait()`, `notify()`, and `notifyAll()` methods are crucial for managing thread synchronization in Java. They allow threads to communicate about the state of shared resources,

enabling more efficient and responsive applications. Understanding how to use these methods effectively is essential for any Java developer working with multi-threading.

If you have any further questions or need clarification on specific points, feel free to ask!

Java Deadlock: Explanation and Examples

Deadlock in Java occurs when two or more threads are blocked forever, waiting for each other to release resources. This situation arises in multithreaded programming when threads hold locks on resources and simultaneously wait for locks held by other threads, creating a cycle of dependencies.

Key Concepts of Deadlock

1. Conditions for Deadlock:

- **Mutual Exclusion:** At least one resource must be held in a non-shareable mode; that is, only one thread can use the resource at any given time.
- **Hold and Wait:** A thread holding at least one resource is waiting to acquire additional resources that are currently being held by other threads.
- **No Preemption:** Resources cannot be forcibly taken from a thread; they must be voluntarily released.
- **Circular Wait:** There exists a set of threads such that each thread is waiting for a resource held by the next thread in the cycle.

2. **Detection:** Deadlocks can be detected using thread dumps or monitoring tools that analyze the state of threads and their resource locks.

3. **Prevention:** Strategies to prevent deadlocks include avoiding nested locks, acquiring locks in a consistent order, and using timeouts.

Example of Deadlock in Java

Here's a simple example demonstrating a deadlock situation in Java:

```
java
VerifyOpen In EditorRunCopy code
1class Resource {
2    private String name;
3
4    public Resource(String name) {
5        this.name = name;
6    }
7
8    public String getName() {
9        return name;
10    }
11}
12
13class Thread1 extends Thread {
14    private Resource resource1;
15    private Resource resource2;
16
17    public Thread1(Resource resource1, Resource resource2) {
18        this.resource1 = resource1;
```

```

19         this.resource2 = resource2;
20     }
21
22     public void run() {
23         synchronized (resource1) {
24             System.out.println(Thread.currentThread().getName() + " locked " +
resource1.getName());
25             try {
26                 // Simulating some work with resource1
27                 Thread.sleep(100);
28             } catch (InterruptedException e) {
29                 e.printStackTrace();
30             }
31             synchronized (resource2) {
32                 System.out.println(Thread.currentThread().getName() + " locked
" + resource2.getName());
33             }
34         }
35     }
36 }
37
38 class Thread2 extends Thread {
39     private Resource resource1;
40     private Resource resource2;
41
42     public Thread2(Resource resource1, Resource resource2) {
43         this.resource1 = resource1;
44         this.resource2 = resource2;
45     }
46
47     public void run() {
48         synchronized (resource2) {
49             System.out.println(Thread.currentThread().getName() + " locked " +
resource2.getName());
50             try {
51                 // Simulating some work with resource2
52                 Thread.sleep(100);
53             } catch (InterruptedException e) {
54                 e.printStackTrace();
55             }
56             synchronized (resource1) {
57                 System.out.println(Thread.currentThread().getName() + " locked
" + resource1.getName());
58             }
59         }
60     }
61 }
62
63 public class DeadlockExample {
64     public static void main(String[] args) {
65         Resource resource1 = new Resource("Resource 1");
66         Resource resource2 = new Resource("Resource 2");
67
68         Thread1 t1 = new Thread1(resource1, resource2);
69         Thread2 t2 = new Thread2(resource1, resource2);
70
71         t1.start();
72         t2.start();
73     }
74 }

```


Explanation of the Example:

- **Thread1** locks resource1 and then tries to lock resource2.
- **Thread2** locks resource2 and then tries to lock resource1.
- This creates a circular wait condition, leading to a deadlock where both threads are waiting indefinitely for each other to release the locks.

Output

When you run the above code, you may see output similar to:

```
VerifyOpen In EditorRunCopy code
1Thread-0 locked Resource 1
2Thread-1 locked Resource 2
```

After this, the program will hang indefinitely because both threads are waiting for each other to release their locks.

Detecting Deadlocks

To detect deadlocks in a running Java application, you can use tools like:

- **Thread Dumps:** You can generate a thread dump using tools like `jstack` or `VisualVM`. The thread dump will show the state of all threads and can help identify deadlocks.
- **Java Monitoring Tools:** Tools like `JConsole` or `VisualVM` can provide insights into thread states and resource locks.

Preventing Deadlocks

1. **Lock Ordering:** Always acquire locks in a consistent order across all threads.
2. **Avoid Nested Locks:** Minimize the use of nested locks to reduce the chances of deadlock.
3. **Use Timeouts:** Implement timeouts when trying to acquire locks, allowing threads to back off and retry instead of waiting indefinitely.

Conclusion

Deadlocks are a critical issue in multithreaded programming that can lead to application hangs and resource wastage. Understanding the conditions that lead to deadlocks, how to detect them, and strategies to prevent them is essential for developing robust Java applications. If you have any further questions or need more examples, feel free to ask!

Java Producer-Consumer Problem: Explanation and Examples

The **Producer-Consumer Problem** is a classic synchronization problem in concurrent programming. It involves two types of processes: producers, which generate data, and consumers, which use that data. The challenge is to ensure that the producer does not add data when the buffer is full and that the consumer does not try to consume data when the buffer is empty.

Key Concepts

- **Buffer:** A shared resource (often a queue) that holds data produced by the producer and consumed by the consumer.
- **Synchronization:** Mechanisms to ensure that the producer and consumer do not access the buffer simultaneously, preventing data inconsistency.
- **Blocking:** The producer should block when the buffer is full, and the consumer should block when the buffer is empty.

1. Implementation Using wait() and notify()

This approach uses Java's built-in synchronization mechanisms to manage access to the shared buffer.

1.1 Buffer Class

```
java
VerifyOpen In EditorRunCopy code
1import java.util.LinkedList;
2
3class Buffer {
4    private final LinkedList<Integer> queue = new LinkedList<>();
5    private final int capacity;
6
7    public Buffer(int capacity) {
8        this.capacity = capacity;
9    }
10
11    public synchronized void produce(int value) throws InterruptedException {
12        while (queue.size() == capacity) {
13            wait(); // Wait if the buffer is full
14        }
15        queue.add(value);
16        System.out.println("Produced: " + value);
17        notifyAll(); // Notify consumers that an item has been produced
18    }
19
20    public synchronized int consume() throws InterruptedException {
21        while (queue.isEmpty()) {
22            wait(); // Wait if the buffer is empty
23        }
24        int value = queue.removeFirst();
25        System.out.println("Consumed: " + value);
26        notifyAll(); // Notify producers that an item has been consumed
27        return value;
28    }
29}
```

1.2 Producer Class

```
java
VerifyOpen In EditorRunCopy code
1class Producer extends Thread {
2    private final Buffer buffer;
3
4    public Producer(Buffer buffer) {
5        this.buffer = buffer;
6    }
7
8    @Override
```

```

9     public void run() {
10         for (int i = 0; i < 10; i++) {
11             try {
12                 buffer.produce(i);
13                 Thread.sleep(100); // Simulate time taken to produce
14             } catch (InterruptedException e) {
15                 e.printStackTrace();
16             }
17         }
18     }
19}

```

1.3 Consumer Class

```

java
VerifyOpen In EditorRunCopy code
1class Consumer extends Thread {
2    private final Buffer buffer;
3
4    public Consumer(Buffer buffer) {
5        this.buffer = buffer;
6    }
7
8    @Override
9    public void run() {
10        for (int i = 0; i < 10; i++) {
11            try {
12                buffer.consume();
13                Thread.sleep(150); // Simulate time taken to consume
14            } catch (InterruptedException e) {
15                e.printStackTrace();
16            }
17        }
18    }
19}

```

1.4 Main Class

```

java
VerifyOpen In EditorRunCopy code
1public class ProducerConsumerExample {
2    public static void main(String[] args) {
3        Buffer buffer = new Buffer(5); // Buffer capacity of 5
4        Producer producer = new Producer(buffer);
5        Consumer consumer = new Consumer(buffer);
6
7        producer.start();
8        consumer.start();
9    }
10}

```

2. Implementation Using BlockingQueue

Java provides a more straightforward way to handle the producer-consumer problem using the `BlockingQueue` interface from the `java.util.concurrent` package. This approach simplifies synchronization and blocking.

2.1 Producer Class

```

java
VerifyOpen In EditorRunCopy code

```

```

1import java.util.concurrent.BlockingQueue;
2import java.util.concurrent.LinkedBlockingQueue;
3
4class Producer implements Runnable {
5    private final BlockingQueue<Integer> queue;
6
7    public Producer(BlockingQueue<Integer> queue) {
8        this.queue = queue;
9    }
10
11    @Override
12    public void run() {
13        for (int i = 0; i < 10; i++) {
14            try {
15                queue.put(i); // This will block if the queue is full
16                System.out.println("Produced: " + i);
17                Thread.sleep(100); // Simulate time taken to produce
18            } catch (InterruptedException e) {
19                Thread.currentThread().interrupt();
20            }
21        }
22    }
23}

```

2.2 Consumer Class

```

java
VerifyOpen In EditorRunCopy code
1class Consumer implements Runnable {
2    private final BlockingQueue<Integer> queue;
3
4    public Consumer(BlockingQueue<Integer> queue) {
5        this.queue = queue;
6    }
7
8    @Override
9    public void run() {
10        for (int i = 0; i < 10; i++) {
11            try {
12                int value = queue.take(); // This will block if the queue is
empty
13                System.out.println("Consumed: " + value);
14                Thread.sleep(150); // Simulate time taken to consume
15            } catch (InterruptedException e) {
16                Thread.currentThread().interrupt();
17            }
18        }
19    }
20}

```

2.3 Main Class

```

java
VerifyOpen In EditorRunCopy code
1import java.util.concurrent.BlockingQueue;
2import java.util.concurrent.LinkedBlockingQueue;
3
4public class BlockingQueueExample {
5    public static void main(String[] args) {
6        BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(5); // Buffer
capacity of 5
7        Thread producerThread = new Thread(new Producer(queue));
8        Thread consumerThread = new Thread(new Consumer(queue));

```

```
9
10     producerThread.start();
11     consumerThread.start();
12 }
13 }
```

Conclusion

The Producer-Consumer problem is a fundamental concept in concurrent programming, illustrating the need for synchronization when multiple threads access shared resources. The first implementation using `wait()` and `notify()` provides a manual approach to synchronization, while the second implementation using `BlockingQueue` leverages Java's concurrency utilities for a more straightforward and efficient solution. Understanding both methods is essential for developing robust multi-threaded applications in Java.

Final Answer

Java Synchronization, Deadlock, Wait, Notify, NotifyAll Methods, and Producer-Consumer Problem: Detailed Explanation with Examples

In this response, I will provide a comprehensive overview of Java synchronization, deadlock, the `wait()`, `notify()`, and `notifyAll()` methods, and the Producer-Consumer problem. Each section will include key points, subpoints, and suitable examples to enhance your understanding.

1. Java Synchronization

Synchronization in Java is a mechanism that ensures that only one thread can access a shared resource at a time. This is crucial in multi-threaded applications to prevent data inconsistency and corruption.

1.1 Why Use Synchronization?

- **Prevent Thread Interference:** Ensures that multiple threads do not interfere with each other when accessing shared resources.
- **Avoid Concurrency Problems:** Mitigates issues like race conditions.

1.2 Types of Synchronization

1. **Method Synchronization:** Declaring a method as synchronized using the `synchronized` keyword.

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1class SynchronizedCounter {
2    private int count = 0;
3
4    public synchronized void increment() {
5        count++;
6    }
7}
```

```

6    }
7
8    public synchronized int value() {
9        return count;
10   }
11}

```

2. **Block Synchronization:** Synchronizing a specific block of code within a method.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class Counter {
2    private int count = 0;
3
4    public void increment() {
5        synchronized (this) {
6            count++;
7        }
8    }
9}

```

3. **Static Synchronization:** Synchronizing static methods to control access to class-level data.

- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1class StaticCounter {
2    private static int count = 0;
3
4    public static synchronized void increment() {
5        count++;
6    }
7}

```

2. **Inter-Thread Communication: wait(), notify(), notifyAll()**

Java provides methods for inter-thread communication, allowing threads to communicate about the state of shared resources.

2.1 Overview of wait(), notify(), and notifyAll()

- **wait():** Causes the current thread to wait until another thread invokes `notify()` or `notifyAll()` on the same object.
- **notify():** Wakes up a single thread that is waiting on the object's monitor.
- **notifyAll():** Wakes up all threads that are waiting on the object's monitor.

2.2 Key Points

- These methods must be called from within a synchronized context.
- They help reduce CPU usage by avoiding busy-waiting.

2.3 Example Implementation

Shared Resource Class:

```

java
VerifyOpen In EditorRunCopy code
1class SharedResource {
2    private int data;
3
4    public synchronized void produce(int value) {
5        data = value;
6        notify(); // Notify waiting threads
7    }
8
9    public synchronized int consume() throws InterruptedException {
10        while (data == 0) {
11            wait(); // Wait until data is produced
12        }
13        return data;
14    }
15}

```

Waiter Class:

```

java
VerifyOpen In EditorRunCopy code
1class Waiter implements Runnable {
2    private SharedResource resource;
3
4    public Waiter(SharedResource resource) {
5        this.resource = resource;
6    }
7
8    @Override
9    public void run() {
10        try {
11            System.out.println(Thread.currentThread().getName() + " waiting to
consume");
12            int value = resource.consume();
13            System.out.println(Thread.currentThread().getName() + " consumed:
" + value);
14        } catch (InterruptedException e) {
15            e.printStackTrace();
16        }
17    }
18}

```

Notifier Class:

```

java
VerifyOpen In EditorRunCopy code
1class Notifier implements Runnable {
2    private SharedResource resource;
3
4    public Notifier(SharedResource resource) {
5        this.resource = resource;
6    }
7
8    @Override
9    public void run() {
10        try {
11            Thread.sleep(1000); // Simulate work
12            resource.produce(42); // Produce data
13            System.out.println(Thread.currentThread().getName() + " produced
data");
14        } catch (InterruptedException e) {
15            e.printStackTrace();

```

```

16     }
17 }
18}

```

Test Class:

```

java
VerifyOpen In EditorRunCopy code
1public class WaitNotifyTest {
2    public static void main(String[] args) {
3        SharedResource resource = new SharedResource();
4        Thread waiter = new Thread(new Waiter(resource), "Waiter");
5        Thread notifier = new Thread(new Notifier(resource), "Notifier");
6
7        waiter.start();
8        notifier.start();
9    }
10}

```

3. Deadlock

Deadlock occurs when two or more threads are blocked forever, waiting for each other to release resources.

3.1 Conditions for Deadlock

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
2. **Hold and Wait:** A thread holding at least one resource is waiting to acquire additional resources.
3. **No Preemption:** Resources cannot be forcibly taken from a thread.
4. **Circular Wait:** A set of threads are waiting for each other in a circular chain.

3.2 Example of Deadlock

```

java
VerifyOpen In EditorRunCopy code
1class Resource {
2    private String name;
3
4    public Resource(String name) {
5        this.name = name;
6    }
7
8    public String getName() {
9        return name;
10    }
11}
12
13class Thread1 extends Thread {
14    private Resource resource1;
15    private Resource resource2;
16
17    public Thread1(Resource resource1, Resource resource2) {
18        this.resource1 = resource1;
19        this.resource2 = resource2;
20    }
21
22    public void run() {

```



```

23     synchronized (resource1) {
24         System.out.println(Thread.currentThread().getName() + " locked " +
resource1.getName());
25         try {
26             Thread.sleep(100);
27         } catch (InterruptedException e) {
28             e.printStackTrace();
29         }
30         synchronized (resource2) {
31             System.out.println(Thread.currentThread().getName() + " locked
" + resource2.getName());
32         }
33     }
34 }
35}
36
37class Thread2 extends Thread {
38     private Resource resource1;
39     private Resource resource2;
40
41     public Thread2(Resource resource1, Resource resource2) {
42         this.resource1 = resource1;
43         this.resource2 = resource2;
44     }
45
46     public void run() {
47         synchronized (resource2) {
48             System.out.println(Thread.currentThread().getName() + " locked " +
resource2.getName());
49             try {
50                 Thread.sleep(100);
51             } catch (InterruptedException e) {
52                 e.printStackTrace();
53             }
54             synchronized (resource1) {
55                 System.out.println(Thread.currentThread().getName() + " locked
" + resource1.getName());
56             }
57         }
58     }
59}
60
61public class DeadlockExample {
62     public static void main(String[] args) {
63         Resource resource1 = new Resource("Resource 1");
64         Resource resource2 = new Resource("Resource 2");
65
66         Thread1 t1 = new Thread1(resource1, resource2);
67         Thread2 t2 = new Thread2(resource1, resource2);
68
69         t1.start();
70         t2.start();
71     }
72}

```

4. Producer-Consumer Problem

The **Producer-Consumer Problem** is a classic synchronization problem involving two types of processes: producers (which generate data) and consumers (which use that data).

4.1 Key Concepts

- **Buffer:** A shared resource (often a queue) that holds data produced by the producer and consumed by the consumer.
- **Blocking:** The producer should block when the buffer is full, and the consumer should block when the buffer is empty.

4.2 Implementation Using wait() and notify()

Buffer Class:

```
java
VerifyOpen In EditorRunCopy code
1import java.util.LinkedList;
2
3class Buffer {
4    private final LinkedList<Integer> queue = new LinkedList<>();
5    private final int capacity;
6
7    public Buffer(int capacity) {
8        this.capacity = capacity;
9    }
10
11    public synchronized void produce(int value) throws InterruptedException {
12        while (queue.size() == capacity) {
13            wait(); // Wait if the buffer is full
14        }
15        queue.add(value);
16        System.out.println("Produced: " + value);
17        notifyAll(); // Notify consumers
18    }
19
20    public synchronized int consume() throws InterruptedException {
21        while (queue.isEmpty()) {
22            wait(); // Wait if the buffer is empty
23        }
24        int value = queue.removeFirst();
25        System.out.println("Consumed: " + value);
26        notifyAll(); // Notify producers
27        return value;
28    }
29}
```

Producer Class:

```
java
VerifyOpen In EditorRunCopy code
1class Producer extends Thread {
2    private final Buffer buffer;
3
4    public Producer(Buffer buffer) {
5        this.buffer = buffer;
6    }
7
8    @Override
9    public void run() {
10        for (int i = 0; i < 10; i++) {
11            try {
12                buffer.produce(i);
13                Thread.sleep(100); // Simulate time taken to produce
14            } catch (InterruptedException e) {
15                e.printStackTrace();
16            }
17        }
18    }
19}
```

```

16         }
17     }
18 }
19}

```

Consumer Class:

```

java
VerifyOpen In EditorRunCopy code
1class Consumer extends Thread {
2    private final Buffer buffer;
3
4    public Consumer(Buffer buffer) {
5        this.buffer = buffer;
6    }
7
8    @Override
9    public void run() {
10        for (int i = 0; i < 10; i++) {
11            try {
12                buffer.consume();
13                Thread.sleep(150); // Simulate time taken to consume
14            } catch (InterruptedException e) {
15                e.printStackTrace();
16            }
17        }
18    }
19}

```

Main Class:

```

java
VerifyOpen In EditorRunCopy code
1public class ProducerConsumerExample {
2    public static void main(String[] args) {
3        Buffer buffer = new Buffer(5); // Buffer capacity of 5
4        Producer producer = new Producer(buffer);
5        Consumer consumer = new Consumer(buffer);
6
7        producer.start();
8        consumer.start();
9    }
10}

```

Conclusion

This comprehensive overview of Java synchronization, deadlock, the `wait()`, `notify()`, and `notifyAll()` methods, and the Producer-Consumer problem provides a solid foundation for understanding concurrency in Java. Each section includes examples to illustrate the concepts effectively. If you have any further questions or need clarification on specific topics, feel free to ask!